

Revisão de Estrutura de Árvores

Aluno – Isaque Kirmse Mendonça Soares

Disciplina - Estrutura de Dados II

Professora: Kadidja Valéria

Etapa 1 — Revisão bibliográfica

Conceitos básicos de árvores

Uma árvore é uma estrutura de dados não linear que organiza elementos de forma hierárquica. Ela é formada por nós que possuem relações entre si.

O nó raiz é o primeiro nó da árvore. Os nós ligados abaixo dele são seus filhos, enquanto o nó que está acima de outro é chamado de pai. Um nó que não possui filhos é chamado de folha.

A altura de uma árvore está relacionada ao maior caminho entre a raiz e uma folha. Os percursos são formas de visitar os nós da árvore, como pré-ordem, em-ordem e pós-ordem.

Referências: Cormen et al. (2022); Szwarcfiter e Markenzon (2010).

Árvore geral

Uma árvore geral é uma estrutura em que um nó pode possuir vários filhos. Diferentemente de uma árvore binária, não existe uma limitação de apenas dois filhos por nó.

Esse tipo de estrutura pode ser utilizado para representar informações hierárquicas, como diretórios e subdiretórios de um sistema de arquivos.

Referência: Szwarcfiter e Markenzon (2010).

Árvore binária

Na árvore binária, cada nó pode possuir no máximo dois filhos, normalmente chamados de filho esquerdo e filho direito.

Ela é uma estrutura básica para diversas outras árvores, como a árvore binária de busca.

Referências: Cormen et al. (2022); Szwarcfiter e Markenzon (2010).

Revisão de Estrutura de Árvores

Árvore binária de busca (ABB)

A árvore binária de busca organiza os valores de forma que os valores menores que um nó ficam na subárvore esquerda e os valores maiores ficam na subárvore direita.

Essa propriedade facilita a busca, inserção e remoção de valores. Entretanto, dependendo da ordem em que os valores são inseridos, a árvore pode ficar muito desequilibrada.

Referências: Cormen et al. (2022); Sedgewick e Wayne (2011).

Árvore AVL

A árvore AVL é uma árvore binária de busca que procura manter seu balanceamento.

Para cada nó, a diferença entre as alturas das subárvores esquerda e direita deve permanecer limitada. Quando uma inserção ou remoção causa um desequilíbrio, podem ser realizadas rotações para reorganizar a árvore.

As rotações podem ser simples ou duplas, dependendo do tipo de desequilíbrio.

Referências: Sedgewick e Wayne (2011); Szwarcfiter e Markenzon (2010).

Árvore rubro-negra

A árvore rubro-negra é uma árvore binária de busca que utiliza uma propriedade adicional de cores. Cada nó possui uma cor, vermelha ou preta.

Existem regras relacionadas a essas cores que ajudam a manter a altura da árvore controlada. Após inserções ou remoções, podem ser necessárias rotações e recolorações.

Referência: Cormen et al. (2022).

Árvore B

A árvore B é uma árvore de busca que permite armazenar várias chaves em um mesmo nó e possuir vários filhos.

Quando um nó fica cheio durante uma inserção, pode ocorrer uma divisão desse nó. Essa característica permite manter a árvore com altura relativamente baixa mesmo quando existem muitos dados.

É muito utilizada em estruturas relacionadas a armazenamento e sistemas de arquivos.

Referências: Cormen et al. (2022); Sedgewick e Wayne (2011).

Revisão de Estrutura de Árvores

Árvore B+

A árvore B+ é uma variação da árvore B em que os dados ficam concentrados nas folhas, enquanto os nós internos funcionam principalmente como índices para direcionar as buscas.

As folhas podem ser ligadas entre si, o que facilita percorrer vários registros em sequência e realizar consultas por intervalo.

Referência: Szwarcfiter e Markenzon (2010).

Heap

O heap é uma árvore binária completa utilizada principalmente para implementar filas de prioridade.

No max-heap, o maior elemento fica na raiz. No min-heap, o menor elemento fica na raiz.

Quando um elemento é inserido ou removido, a estrutura pode precisar ser reorganizada para manter a propriedade do heap.

Referências: Cormen et al. (2022); Szwarcfiter e Markenzon (2010).

Trie

A trie é uma estrutura utilizada principalmente para armazenar e pesquisar sequências de caracteres.

Os caracteres são organizados ao longo dos caminhos da árvore. Assim, palavras que possuem o mesmo prefixo podem compartilhar parte da estrutura.

Um exemplo de aplicação é a busca de palavras e sistemas de preenchimento automático.

Referência: Sedgewick e Wayne (2011).

Etapa 2 — Quadro comparativo

Estrutura	Organização dos dados	Regra ou propriedade principal	Operação ou ajuste importante	Exemplo de aplicação	Referência consultada
Árvore binária de busca (ABB)	Valores menores ficam à esquerda e maiores à direita de cada nó.	Manter a ordenação dos valores em relação aos nós.	Busca, inserção e remoção seguindo a propriedade da árvore.	Busca de dados ordenados.	Cormen et al. (2022); Sedgewick e Wayne (2011)
AVL	ABB que mantém suas subárvores	A diferença entre as alturas das subárvores	Rotações simples ou duplas após	Sistemas que precisam de buscas	Sedgewick e Wayne (2011); Szwarcfiter e

Revisão de Estrutura de Árvores

	balanceadas.	deve ser limitada.	alterações.	eficientes.	Markenzon (2010)
Rubro-negra	ABB em que cada nó possui uma cor, vermelha ou preta.	Regras de cores ajudam a controlar a altura da árvore.	Rotações e recolorações após inserções ou remoções.	Estruturas que precisam de busca e atualização eficientes.	Cormen et al. (2022)
B	Possui várias chaves e vários filhos em cada nó.	Manter os nós dentro das regras de quantidade de chaves e filhos.	Divisão de nós quando ficam cheios.	Bancos de dados e sistemas de arquivos.	Cormen et al. (2022); Sedgewick e Wayne (2011)
B+	Os nós internos funcionam como índices e os dados ficam nas folhas.	As folhas armazenam os registros e podem ser ligadas.	Divisão e reorganização dos nós.	Índices e consultas por intervalo.	Szwarcfiter e Markenzon (2010)
Árvore geral	Cada nó pode possuir vários filhos.	Não existe limite de dois filhos por nó.	Inserção e remoção conforme a hierarquia.	Diretórios e estruturas hierárquicas.	Szwarcfiter e Markenzon (2010)
Árvore binária	Cada nó possui no máximo dois filhos.	Um filho pode estar à esquerda e outro à direita.	Inserção e remoção de nós.	Estruturas hierárquicas e representação de expressões.	Cormen et al. (2022); Szwarcfiter e Markenzon (2010)
Heap	Árvore binária completa organizada por prioridade.	No max-heap, o pai é maior ou igual aos filhos; no min-heap, o pai é menor ou igual.	Reorganização após inserção ou remoção.	Filas de prioridade e heapsort.	Cormen et al. (2022)
Trie	Dados são organizados por caracteres e prefixos.	Palavras com o mesmo prefixo podem compartilhar nós.	Percorrer caracteres durante busca e inserção.	Busca de palavras e autocomplete.	Sedgewick e Wayne (2011)

Revisão de Estrutura de Árvores

Etapa 3 — Identificação por analogias

1. Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.

Estrutura: Árvore AVL

A situação representa uma árvore AVL porque ela busca manter o equilíbrio entre as subárvores. Quando ocorre um desequilíbrio, podem ser realizadas rotações para reorganizar os nós.

Limite da analogia: uma estante física não realiza cálculos de altura nem possui as regras matemáticas de balanceamento de uma árvore AVL. A comparação representa apenas a ideia de um lado ficar maior que o outro.

2. Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.

Estrutura: Árvore B

A situação representa uma árvore B porque seus nós podem armazenar várias chaves. Quando um nó fica cheio durante uma inserção, ele pode ser dividido para manter a estrutura organizada.

Limite da analogia: uma página de catálogo não possui exatamente as mesmas regras de distribuição de chaves e filhos existentes em uma árvore B.

3. Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.

Estrutura: Heap

A situação representa um max-heap, pois o elemento de maior prioridade fica na raiz e pode ser retirado primeiro.

Limite da analogia: uma fila de tarefas pode ser implementada de outras maneiras. O heap é uma estrutura específica utilizada para implementar filas de prioridade.

4. Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.

Estrutura: Trie

A situação representa uma trie porque ela organiza os elementos de acordo com seus caracteres. Palavras que possuem o mesmo prefixo podem compartilhar os mesmos nós no início do caminho.

Limite da analogia: um índice pode utilizar outras estruturas de dados. A trie é especialmente adequada para trabalhar com caracteres e prefixos.

Revisão de Estrutura de Árvores

5. Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.

Estrutura: Árvore rubro-negra

A situação representa uma árvore rubro-negra porque ela utiliza nós vermelhos e pretos e regras relacionadas às cores para manter a árvore aproximadamente balanceada. Rotações e recolorações podem ser utilizadas após alterações.

Limite da analogia: as cores não representam características dos dados. Elas são informações utilizadas pela própria estrutura para manter suas propriedades.

6. Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.

Estrutura: Árvore B+

A situação representa uma árvore B+ porque os registros são armazenados nas folhas e os nós internos funcionam como índices para direcionar a busca. A ligação entre as folhas facilita a leitura sequencial e as consultas por intervalo.

Limite da analogia: um índice real pode utilizar outras estruturas e mecanismos de armazenamento. A analogia representa a organização característica da árvore B+.

7. Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.

Estrutura: Árvore binária de busca (ABB)

A situação representa uma ABB porque sua principal propriedade é colocar os valores menores que um nó na subárvore esquerda e os valores maiores na subárvore direita.

Limite da analogia: a situação descreve apenas a propriedade de organização da ABB. Ela não representa outras operações, como remoção, nem o problema de uma ABB ficar desbalanceada.

Referências bibliográficas

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.

SEDGEWICK, Robert; WAYNE, Kevin. Algorithms. 4. ed. Boston: Addison-Wesley, 2011.

SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e Seus Algoritmos. 3. ed. Rio de Janeiro: LTC, 2010.

Revisão de Estrutura de Árvores

Auxílio de estruturação e correção com a IA GPT.